PROJET DE MASTER 2 - MDSIA

De la prediction à un service MLOps

Prédiction du churn des clients télécoms

Dossier de présentation complet · Novembre 2026

- Présenté par Gibril KAMADEU

Machine Learning | DevOps | Observabilité

Le défi : anticiper le départ des clients

Du besoin métier à une décision mieux informée

Problème

Le churn représente la perte d'un abonné. Identifier les clients à risque peut aider à mieux cibler les actions de fidélisation.

Réponse proposée

Un modèle de classification binaire estime la probabilité de départ à partir de six caractéristiques client.

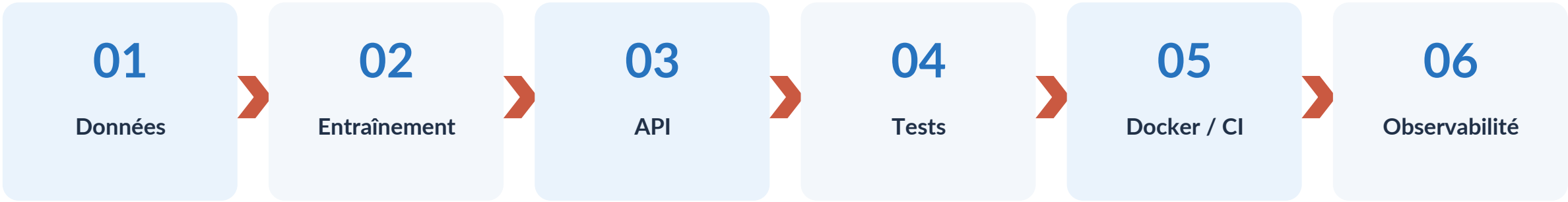
Valeur attendue

Fournir un score exploitable par une application, sans remplacer le jugement humain ni promettre une certitude.

Périmètre : prototype pédagogique fondé sur des données synthétiques.

Notre démarche : une chaîne MLOps complète

Chaque étape transforme une expérimentation en service contrôlable



Objectif : reproductibilité, validation automatisée, traçabilité et suivi du fonctionnement.

Données et variables du modèle

Classification binaire : churn = 1 ou 0

1. Âge

Variable explicative

2. Ancienneté

Variable explicative

3. Revenu

Variable explicative

4. Nombre de réclamations

Variable explicative

5. Montant mensuel

Variable explicative

6. Retards de paiement

Variable explicative

Dataset synthétique • Analyse exploratoire • Séparation stratifiée 80 % / 20 %

Trois modèles entraînés, un modèle exposé

La sélection du Random Forest est impose dans le TP mais il ne présente pas le Meilleur score. On aurait du choisir l'arbre de décision

Régression logistique

StandardScaler + LogisticRegression
max_iter = 1 000

Arbre de décision

Profondeur maximale = 5
random_state = 42

Random Forest

200 arbres
random_state = 42
Artefact : models/model.pkl

```
(.venv) kagibru@KGB:~/projects/ml-churn-project$ python src/train.py

Entraînement : logistic_regression
{'accuracy': 0.625, 'precision': 0.582089552238806, 'recall': 0.45348837209302323, 'f1': 0.5098039215686274}

Entraînement : decision_tree
{'accuracy': 0.6325, 'precision': 0.5827814569536424, 'recall': 0.5116279069767442, 'f1': 0.5448916408668873}

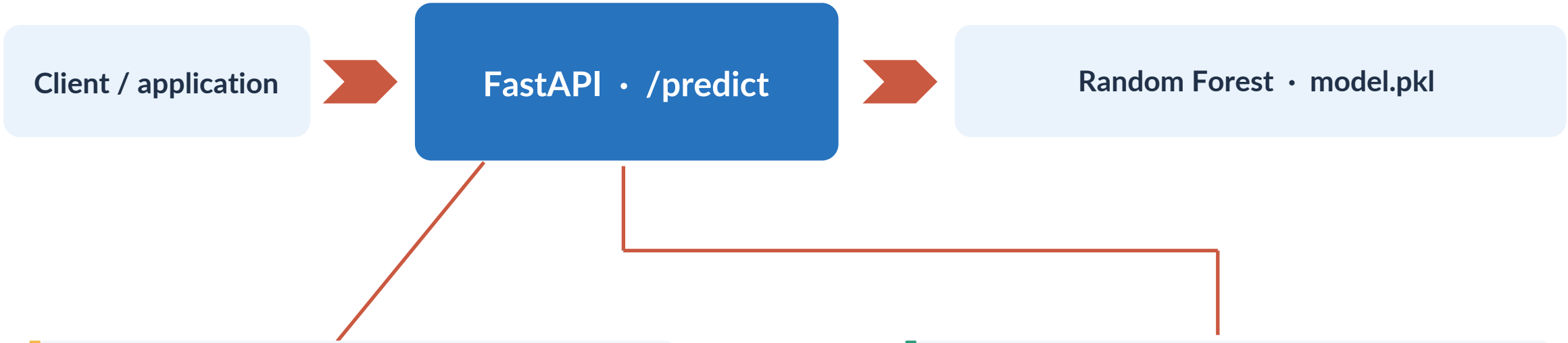
Entraînement : random_forest
{'accuracy': 0.605, 'precision': 0.5514705882352942, 'recall': 0.436046511627907, 'f1': 0.487012987012987}
```

Évaluation sur le jeu de test : accuracy, précision, rappel et F1-score.

Limite : pas de sélection automatique du meilleur modèle ni de validation croisée.

Architecture fonctionnelle

Un service de prédiction connecté à deux chaînes d'observabilité



Journaux

Logstash → Elasticsearch → Kibana

Métriques

/metrics → Prometheus → Grafana

API de prédiction : un contrat simple

Le modèle devient consommable par d'autres applications

Entrée · POST /predict

age · anciennete · revenu
nb_reclamations · montant_mensuel
retards_paiement

Sortie · JSON

prediction : CHURN / NO_CHURN
probability : score estimé
model_version : 1.0.0
duration_ms : durée côté application

Routes complémentaires : /health · /model/info · /metrics

Conteneurisation et intégration continue

Un environnement local reproductible et des contrôles automatiques

FastAPI 8000	Elasticsearch 9200	Logstash interne
Kibana 5601	Prometheus 9090	Grafana 3001

GitHub Actions : tests Pytest → construction de l'image Docker

Vérifications réalisées

La validation couvre le service, l'intégration et l'observabilité

4 / 4 Tests API Pytest	**1 / 1** Test Selenium	**OK** Build Docker CI
OK ELK : logs consultables	**UP = 1** Cible Prometheus	**OK** Dashboard Grafana

Preuves de validation archivées dans docs/validation/.

JMeter : la charge augmente, le débit plafonne

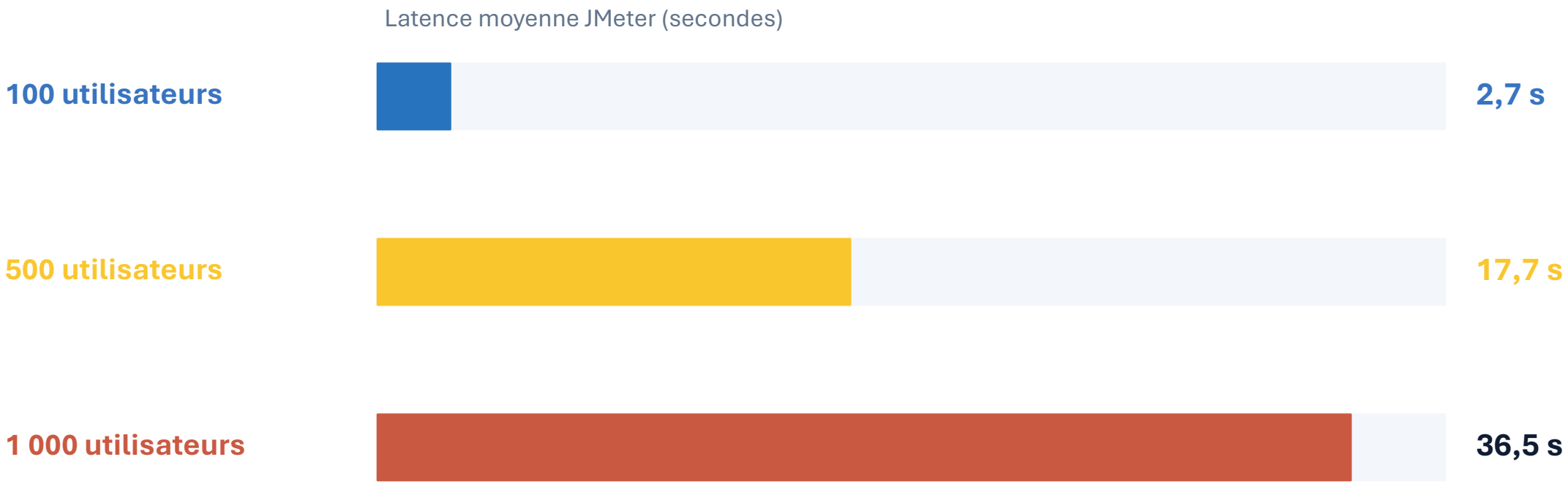
Trois batch de 1 000, 5 000 et 10 000 requêtes

Utilisateurs	Latence moy.	P95	Débit (req/s)	Erreurs
100	2,7 s	3,8 s	26,4	0
500	17,7 s	19,9 s	25,6	0
1 000	36,5 s	39,9 s	25,5	0

0 erreur ≠ bonne performance : la latence devient élevée sous forte concurrence.

Lecture des performances

Une limite de capacité apparaît dans l'environnement testé



Débit quasi stable : environ 25–26 requêtes par seconde.

Observabilité : voir et comprendre le fonctionnement

Les logs et les métriques apportent des perspectives complémentaires

ELK · Traçabilité

L'API produit des événements. Logstash les reçoit, Elasticsearch les indexe et Kibana permet leur exploration.

Attention : les erreurs d'envoi vers Logstash peuvent entraîner une perte de traces.

Prometheus + Grafana · Pilotage

Prometheus collecte /metrics. Grafana affiche le volume de requêtes, les erreurs et les durées de réponse.

Des seuils d'alerte et des objectifs de service restent à définir.

Une métrique signale une anomalie ; les journaux aident à en rechercher la cause.

Maturité : une industrialisation partielle

Ce qui fonctionne déjà et ce qui manque pour la production

Acquis du prototype

- API de prédiction opérationnelle
- Tests automatisés et build CI
- Six services Docker Compose
- Campagnes JMeter documentées
- Logs ELK et métriques Grafana
- Documentation et preuves de validation

Évolutions nécessaires

- Sécurité, authentification, secrets
- Critères objectifs de choix du modèle
- Versionnement des données et modèles
- Déploiement et rollback automatisés
- Alertes et objectifs de disponibilité
- Résilience et optimisation sous charge

Feuille de route et conclusion

Passer d’un laboratoire validé à un service exploitable

1	**Fiabiliser**	Sécurité, validation des entrées, diagnostic de la latence
2	**Automatiser**	Registre d’images, déploiement, retour arrière
3	**Gouverner le ML**	Traçabilité des données, suivi des modèles, dérive
4	**Exploiter**	Alertes, sauvegardes, procédures d’incident

Résultat : une chaîne MLOps fonctionnelle, démontrée — pas encore prête pour la production.

DÉPÔT GIT — VUE D'ENSEMBLE DU PROJET

ml-churn-project • GitHub : kagibru/ml-churn-project

Organisation du dépôt

data/	Jeu de données synthétique et graphiques
notebooks/	Analyse exploratoire Jupyter
src/ + models/	Génération, entraînement et modèle sérialisé
api/ + tests/	FastAPI, Pytest et Selenium
monitoring/	Prometheus, Grafana et pipeline ELK
performance/jmeter/	Plan et résultats des tests de charge
docs/ + presentation/	Architecture, rapports, preuves et présentation
.github/ + Dockerfile + docker-compose.yml	CI, image et orchestration des services

Cycle de livraison

Branche feature → Pull Request → develop / main

GitHub Actions : tests API → build Docker

Ce que le dépôt permet de reproduire

1. Générer les données et entraîner les modèles
2. Démarrer l'API et les six services Docker
3. Exécuter les tests et consulter les métriques
4. Vérifier les résultats et les preuves de validation

Un dépôt unique pour le code, les expérimentations, le déploiement, l'observabilité et les livrables.

Data Science

Justification du problème, de la cible et de la méthode de validation

Pourquoi prédire le churn ?

Un opérateur peut identifier les abonnés présentant un risque de départ et prioriser des actions de fidélisation. Le score éclaire la décision ; il ne prouve pas la cause du départ.

Quelle est la variable cible ?

churn : 1 = départ, 0 = maintien. Les six autres colonnes (âge, ancienneté, revenu, réclamations, montant mensuel, retards) sont les variables explicatives ou les caractéristiques.

Pourquoi une classification ?

La cible prend deux modalités discrètes. On prédit une classe et, via predict_proba, un score associé à la classe positive, plutôt qu'un montant continu.

Comment le jeu a-t-il été séparé ?

train_test_split : 80 % entraînement / 20 % test, random_state=42 et stratify=y pour conserver approximativement la proportion des classes.

Choix du modèle · Ce que montrent les expériences

Un choix pédagogique documenté, et non une victoire automatique du Random Forest

Quels modèles ont été comparés ?

Régression logistique (avec StandardScaler), arbre de décision (profondeur 5) et Random Forest (200 arbres). Évaluation sur le même jeu de test.

Pourquoi Random Forest ?

Ensemble d'arbres capable de capturer des interactions non linéaires et de limiter la variance d'un arbre isolé. Il est imposé comme modèle exposé dans le TP bien qu'il ne fournisse pas le meilleure score.

Est-ce le meilleur modèle observé ?

Non. Résultats locaux : F1 régression logistique ≈ 0,510 ; arbre ≈ 0,545 ; Random Forest ≈ 0,487. L'arbre est meilleur sur ce critère ; la sélection RF n'est pas automatisée.

Que faudrait-il améliorer ?

Validation croisée, recherche d'hyperparamètres, comparaison précision/rappel/F1/AUC, seuil métier, données réelles et analyse de stabilité avant toute sélection de production.

API et tests

Pourquoi exposer le modèle et comment prouver son fonctionnement ?

Pourquoi une API et pourquoi FastAPI ?

L'API découple le modèle des applications consommatrices. FastAPI fournit validation des entrées via Pydantic, documentation Swagger et endpoints HTTP simples.

Différence entre /health et /predict ?

GET /health indique que le service répond (status UP). POST /predict accepte six variables et renvoie la classe, la probabilité, la version et la durée de traitement.

Pourquoi Pytest et Selenium ?

Pytest vérifie automatiquement le contrat des endpoints (4 tests réussis). Selenium pilote un navigateur pour vérifier la documentation Web Swagger (1 test réussi localement).

Unitaire, intégration et E2E ?

Unitaire : une fonction isolée. Intégration : plusieurs composants ensemble, p. ex. route + modèle. E2E : parcours visible de bout en bout dans un navigateur. Le test Selenium ici reste limité à Swagger.

Performance

Interprétation des trois campagnes JMeter

Pourquoi utiliser JMeter ?

Pour générer des requêtes HTTP concurrentes et mesurer le débit, les temps de réponse, les percentiles et les erreurs sous charge.

Charge, stress et endurance ?

Charge : comportement sous trafic attendu. Stress : pousser au-delà de la capacité normale. Endurance : maintenir une charge prolongée pour révéler fuites ou dégradations. Nos campagnes ne sont pas un test d'endurance.

Que signifie le P95 ?

95 % des réponses ont une durée inférieure ou égale à cette valeur, dans l'échantillon mesuré. À 1 000 utilisateurs, P95 ≈ 39,9 s : 5 % sont encore plus lentes.

Que conclure des résultats ?

100/500/1 000 utilisateurs : 1 000/5 000/10 000 requêtes, 0 erreur observée. Débit ≈ 26,4/25,6/25,5 req/s ; latence moyenne ≈ 2,7/17,7/36,5 s. Saturation probable à investiguer.

Git, Docker, CI/CD

Reproductibilité, collaboration et automatisation

Pourquoi Git et les branches ?

Git conserve l'historique et facilite le retour arrière. main, develop et feature/* isolent les changements ; les Pull Requests favorisent la revue avant fusion.

Pourquoi Docker ?

L'image regroupe application et dépendances. Docker Compose démarre six services coordonnés : API, Elasticsearch, Logstash, Kibana, Prometheus et Grafana.

Pourquoi CI/CD ?

À chaque push ou PR vers main/develop, GitHub Actions installe Python 3.11, exécute les tests API puis construit l'image Docker si les tests réussissent.

Quelles limites du pipeline actuel ?

La construction Docker utilise push:false : ni publication d'image ni déploiement automatisé. Selenium, JMeter et tests de charge ne sont pas exécutés dans le workflow actuel.

Observabilité

Deux approches complémentaires : événements et mesures agrégées

Logs et métriques : quelle différence ?

Les logs retracent des événements précis (requête, résultat, erreur). Les métriques agrègent l'activité et les durées dans le temps ; elles facilitent les alertes et les tendances.

Pourquoi ELK ?

Logstash collecte les événements de FastAPI, Elasticsearch les indexe et Kibana permet de les rechercher. La chaîne a été vérifiée avec des événements de prédiction.

Pourquoi Prometheus et Grafana ?

Prometheus interroge /metrics et conserve des séries temporelles. Grafana interroge Prometheus pour afficher volume, succès, erreurs et latences.

Que surveiller en production ?

Disponibilité, taux d'erreurs, P95/P99, débit, saturation CPU/mémoire, dérive des données, distribution des scores, qualité des prédictions après obtention des vraies étiquettes.

Cycle de vie MLOps

Ce qui est implémenté aujourd'hui et ce qui reste à industrialiser

Comment versionner un modèle ?

Actuellement, model.pkl est sauvegardé et l'API annonce la version 1.0.0. Pour la production : registre de modèles, métadonnées (dataset, code, paramètres, métriques) et artefacts immuables.

Et si un nouveau modèle est moins performant ?

Ne pas le promouvoir. Comparer sur un jeu de validation indépendant et des critères métier ; conserver la version active et documenter le résultat de l'évaluation.

Comment détecter une dégradation ?

Surveiller dérive des variables et des scores, puis précision/rappel/F1 lorsque les vraies sorties deviennent disponibles. Définir seuils d'alerte et procédures de réentraînement.

Plusieurs versions et retour arrière ?

Conserver des versions identifiées dans un registre, déployer de façon progressive (canary/blue-green) et rétablir l'image et l'artefact précédents si les contrôles échouent.

Bilan final · Ce qui est prouvé et ce qui reste ouvert

Une chaîne pédagogique MLOps fonctionnelle, sans confusion avec une plateforme de production

Démonstrations réalisées

Dataset et entraînement ; API opérationnelle ; 4 tests API et 1 Selenium réussis ; build Docker CI ; JMeter à 100/500/1 000 utilisateurs ; journaux ELK et supervision Prometheus/Grafana.

Résultats clés

La chaîne de prédiction fonctionne ; aucune erreur HTTP observée pendant les campagnes JMeter. Le débit plafonne autour de 25–26 req/s et les temps de réponse augmentent fortement.

Limites méthodologiques

Dataset synthétique, sélection RF imposée, absence de validation croisée approfondie, pas de suivi de dérive métier ni de registre de modèles ; performances dépendantes du laboratoire local.

Priorités de progression

Sécuriser les services ; optimiser la latence ; renforcer les tests ; publier et déployer automatiquement ; versionner les artefacts ; suivre la qualité ML avec retour terrain.